funnelling

Marco Servetto
School of Engineering and Computer Science, Victoria University of Wellington, New Zealand
Colin S. Gordon
School of Computer & Information Sciences, Drexel University, United States of America

Abstract

...

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Funding *Colin S. Gordon*: US NSF CISE Award CCF-2007582

Acknowledgements I want to thank ...

1 Introduction

Java includes a useful feature called *method-local inner classes*, which has surprisingly subtle consequences. Like anonymous inner classes, this is a way for a method in a Java program to declare a class whose details are defined locally, entirely within the method, when the identity of the class is not needed outside the method's scope. However, these local classes are named, which allows them to be used with other language features exploiting nominal types. For example, one can cast objects to the method-local class type.

Unfortunately, as Servetto et al. [8] pointed out, there is an unsound interaction with generics. A non-generic method-local inner class `MyFoo` written in a generic method, or in a method of a generic class, can implicitly see the generic types in scope. Instances of `MyFoo` can then escape the lifetime of their enclosing method, where the generics had a certain runtime instantiation, and re-enter a second call of the same enclosing method, where the generics could have a different runtime instantiation. Servetto et al. probed the behaviour of JVM languages that support the same feature (Scala and Kotlin) and found that these had the same unsoundness problem for some use cases, but did flag concerns in other cases. However, this was not systematically investigated.

In this paper we clearly articulate the problem they observed, characterize the key properties that lead to the issue, and propose a static solution for future languages.

Example 1 demonstrates the issue.

```
// Example 1
public class Broken {
  static <X> Foo<X> m(Object other, final X x){
    class MyFoo extends Foo<X>{ public X x(){ return x; } }
    if(!(other instanceof MyFoo)){ return new MyFoo(); }
    return ((MyFoo)other); // cast passes with no warnings
  }
  public static void main(String[] a) {
    Foo<Integer> r1= Broken.<Integer>m("anything",5);
    Foo<String> r2= Broken.<String>m(r1,"Hi");
    System.out.println(r1.x()); //5
    System.out.println(r2.x()); //Throws ClassCastException: ..
  }
}
class Foo<X>{ public X x() {return x();} }
```

Line 4 declares a method-local inner class `MyFoo`, which depends on the generic parameter instantiation for `X` when the method is invoked. The `main` method invokes `m` twice. Line 9 invokes it for `Integer`s. Since the `other` argument for that call is not a `MyFoo`, the first invocation returns a `MyFoo` which is a `Foo<Integer>`. This result is then passed back into the second call to `m` on Line 10. This time, the `instanceof` check succeeds, because `other` in this case *is* a `MyFoo`, so the method (successfully) casts the `other` argument to `MyFoo`. Since `MyFoo` is declared as extending `Foo<X>`, this is a valid result for the function. Thus the `Foo<Integer>` that was passed as the first argument of the second call, on Line 10, is returned as a `Foo<String>`.

Clearly, this is wrong, because there are now two aliased variables of incompatible types; consequently, the later code fails. The failure itself is contrived. The two `println` calls actually dispatch to different overloads: the first to a version accepting an `Object`, and the second to the more specific overload expecting a `String`. The second call passes a value from a generic API to the specific overload, and this causes the Java compiler to insert the failing `checkcast` in the generated bytecode.

explain why (raw types)

A different version of Java could stop this problem at four points, two at compile time and two at run time:

- The `instanceof` check could have been statically flagged.
- The cast could have been statically flagged.
- The `instanceof` could have returned false in the second invocation, since it was used on a `MyFoo` from a different generic parameter instantiation.
- The cast could have failed, again because it was used on a `MyFoo` from a different generic parameter instantiation.

The source of this unsoundness is that the Java compiler does not treat `MyFoo` as generic. The generic type is seen as 'in scope', and it is 'captured' exactly as the values of local variables are captured. Within a single execution of the enclosing method, this is safe, because the type is constant. A `MyFoo` instance is conceptually an instance of a class that captures the generic type that is in scope when the instance is created. But when such a `MyFoo` instance escapes one execution of the enclosing method and enters a second execution of the same method, the instance is still logically connected to the same class type, with the original captured generic type. However, both the static type system and the run-time checks of casts and `instanceof` treat it as if it were connected to the new class type, with the new captured generic type.

Generalizing closure-capture behaviour to generic types as described above feels natural, but it is unsound in many situations. Closure capture is intrinsically private: a closure does not show in its external type signature what it has captured. Here it ends up being so private that the type system and the runtime have no access to it either. The static type system has to work on the publicly visible type, and in Java that type is not generic. This makes the compiler assume that the simplest nominal type identity is sufficient for casts and `instanceof` checks.

This problem touches on Java's erasure of generics, but only indirectly: even languages with reified generics store reified generics for the generic arguments exposed to the type system: a type like `List<Foo>` would get one reified generic, `Foo`, together with some language-mediated API to access it. A type like `MyFoo` would get no reified generics, since the generic type would have been captured and not exposed by the type. Thus, we argue that erasure versus reification is only marginally connected to this problem. The conceptual hurdle is in

the distinction between propagating the scope of generic types explicitly or implicitly.[1]

It is tempting to accept this as a quirk of Java, which is already known to have other sources of unsoundness [2]. However, this problem is broader in both scope and impact, as we show in the rest of the paper. This issue affects the statically typed JVM languages that support local classes, including Scala and Kotlin. It also propagates to compilers of those languages that do not target the JVM, such as Scala Native , and to ahead-of-time compilation of JVM bytecode, such as GraalVM's Native Image.

cite Scala Native

In the remainder of this paper:

- We detail the breadth of the problem, and how it is handled by various existing language implementations.
- We develop a core language modelling Java's generics, method-local classes and type tests, and show that typing method-local classes as Java does is unsound.
- We develop a new type system mechanism for systematically tracking dependence of a method-local class on generic parameters, and show that it soundly prevents the problem. Our solution, *funnelling*, makes the dependency of a method-local class on the generic types in scope explicit in its type: in Example 1 above, `MyFoo` would funnel `X` and become `MyFoo<X>`.

2 More Failing Examples

Example 1 is one of a family of programs that Java accepts with no warning and that fail at run time. This section shows more programs of this family.

2.1 Example 2: No Generic Supertype

In Example 1, the local class `MyFoo` mentions `X` in its supertype `Foo<X>`. Example 2 exercises the same fault without needing a generic supertype. Here `MyFoo` extends `Object`, and `X` occurs only in the return type of the method `x()` of `MyFoo`. The method `m` returns `Object`, so no type outside `m` mentions `X`.

```
1  // Example 2
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(!(other instanceof MyFoo)){ return new MyFoo(); }
6      MyFoo local= (MyFoo)other; // cast passes with no warnings
7      System.out.println("old was "+f.apply(local.x())); //ClassCastException
8      return local;
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

[1] Since JDK 16, with `Object o`, `o instanceof Foo<Integer>` is a compile-time error (before JDK 16, every non-reifiable `instanceof` target was an error), while the cast `(Foo<Integer>)o` is an unchecked warning; `instanceof` never warns. Modern Java accepts `o instanceof C<Ts>` only when the static type of `o` determines `Ts`; in this way an erased and a reified runtime give the same answer, as long as the program has no heap pollution. Example 1 creates heap pollution without any warning: `r2` has static type `Foo<String>`, but it refers to a `Foo<Integer>`.

The first call of `m` instantiates `X` with `Integer`, and returns a `MyFoo` whose method `x()` returns `5`. The second call of `m` instantiates `X` with `String`. In the second call, `other instanceof MyFoo` is true, and the cast `(MyFoo)other` at Line 6 succeeds with no warning. Then `local.x()` has static type `String`, but returns the `Integer 5`, and `f.apply(local.x())` at Line 7 throws a `ClassCastException`.

consider moving this level of detail (the cast is in a generated synthetic method) to the appendix. Maybe just say "the cast is in generated code" and put the rest in an appendix? Or footnote?

No instruction of `m` performs the failing cast. `javac` compiles the body of the lambda `s->s` into a private static method `lambda$main$1(String)` of the class `Broken`. At run time, the first evaluation of the lambda `s->s` calls `LambdaMetafactory`, which generates a class implementing `Function`; we call this class the *lambda class*. The method `apply(Object)` of the lambda class casts its argument to `String`, and then calls `lambda$main$1` with the result of that cast. Thus `f.apply(local.x())` passes the `Integer 5` to the method `apply` of the lambda class, and the cast to `String` in that method throws the `ClassCastException`.

The stack trace of the `ClassCastException` hides this. `LambdaMetafactory` defines the lambda class as a hidden class, and the JVM omits the frames of the methods of hidden classes from stack traces, unless the JVM runs with the diagnostic option `-XX:+ShowHiddenFrames`. So the topmost frame of the stack trace is the frame of `m`, at Line 7. The bytecode of `m` at Line 7 does contain a `checkcast` instruction to `String`, on the result of `f.apply`, but that instruction is never reached. The failure is detected in the lambda class: in code that the programmer did not write, at a point that depends on how javac and the JVM implement lambdas.

2.2 Example 3: No Explicit Cast

Example 3 exercises the same fault without needing an explicit cast. Java accepts Example 3 with no warning. Here we rely on the `other instanceof MyFoo local` test to bind `local` in one step.

```
1  // Example 3
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(other instanceof MyFoo local){ // no cast, no warnings
6        System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      }
8      return new MyFoo();
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`.

2.3 Example 4: Pattern Matching in `switch`

Colins ays: this should be one sentence after previous example that a switch can trigger the same thing, and ref to an appendix w/ code

Example 4 exercises the same fault with a `switch` instead of an `instanceof`. Here the case `case MyFoo local` performs the same type test as the pattern `other instanceof MyFoo local` of Example 3.

```
1  // Example 4
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

2.4 Example 5: No `checkcast` on the Result

this is redundant with example 2's lambda discussion. Maybe use this code for three examples: Object vs. generic type, switch instead of explicit instanceof, and lambda moving cast failure to synthesized method. actually the actual lambdas are different. Can we fold example 5 into example 2 by changing line 12 to ""+s.length"?

Example 5 exercises the same fault without a `checkcast` instruction on the result of `f.apply` in the bytecode of `m`. Example 5 could be expressed with either the `switch` of Example 4 or the `instanceof` of Example 3; here we chose the switch, but the other way would make the same point. Example 5 uses `Function<X,Integer>` and the lambda `s->s.length()` instead of `Function<X,String>` and the lambda `s->s`.

```
1  // Example 5
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,Integer> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->i);
16     Broken.<String>m(r1,"Hi",s->s.length());
17   }
18 }
```

The failure is the failure of Example 2: the method `apply` of the lambda class casts its argument, the `Integer 5`, to `String` before calling `lambda$main$1(String)`. The difference is only in the bytecode of `m`. Here the result of `f.apply` has static type `Integer`, and javac passes that result to the string concatenation without casting it to `Integer`, since any object can be converted to a `String`. So the bytecode of `m` contains no `checkcast` instruction on the result of `f.apply`. In Examples 2 to 4, the bytecode of `m` contains a `checkcast` instruction to `String` on the result of `f.apply`, but that instruction is never reached.

javac inserts a `checkcast` instruction where a value of an erased generic type is used at a more specific type. In Example 1, such a `checkcast` instruction detects the failure: javac inserts it in `main`, before the call of the overload `println(String)`. So one could expect a program with no `checkcast` instruction on the result of `f.apply` to fail later, in some virtual call, or not to fail at all. Example 5 shows that this expectation is wrong. In Examples 2 to 5, the value with the wrong type is the argument of `f.apply`: `local.x()` has static type `String`, but returns the `Integer 5`. The method `apply` of the lambda class casts that argument to `String` before `m` receives any result from `f.apply`. Thus the failure is detected at the same point in Examples 2 to 5, whether or not the bytecode of `m` contains a `checkcast` instruction on the result of `f.apply`.

2.5 Example 6: `X` Only in Private Members

Example 6 exercises the same fault without `X` in the public signature of `MyFoo`. Here `MyFoo` extends `Object`, and its only public method `show` takes a `MyFoo` and returns a `String`. `X` occurs only in the type of the private field `v`, and in the body of `show`, through the captured `f`.

focus is not really about private members, but that MyFoo's public signature doesn't mention X. Could have line 6 just capture x rather than storing in a field. Could turn example 2 into a family where we replace one line of example 2 with this show() definition we get this new example where X doesn't appear in the public signature.

```
1  // Example 6
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{
5        private final X v= x;
6        public String show(MyFoo o){ return f.apply(o.v); }
7      }
8      MyFoo mine= new MyFoo();
9      if(other instanceof MyFoo local){ // no cast, no warnings
10       System.out.println("old was "+mine.show(local)); //ClassCastException
11     }
12     return mine;
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

The first call of `m` returns a `MyFoo` whose field `v` holds the `Integer 5`. In the second call of `m`, `mine` is a new `MyFoo` of the second call, and `local` is the `MyFoo` of the first call. At Line 10, `mine.show(local)` applies the `f` of the second call, the lambda `s->s`, to the field `v` of `local`, the `Integer 5`. As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`. No type that a client of `MyFoo` can see mentions `X`, and still the two

instances of `MyFoo` disagree on `X`.

2.6 Counterexample 7: Class Generics

Counterexample 7 moves Example 3 into a generic class: `X` is a generic parameter of the class `Box` instead of a generic parameter of the method `m`.

```
1  // Example 7
2  class Box<X> {
3    final X x;
4    Box(X x){ this.x= x; }
5    Object m(Object other, Function<X,String> f){
6      class MyFoo{ public X x(){ return x; } }
7      if(other instanceof MyFoo local){ // error: Object cannot be safely cast to MyFoo
8        System.out.println("old was "+f.apply(local.x()));
9      }
10     return new MyFoo();
11   }
12 }
13 public class Broken {
14   public static void main(String[] a){
15     Object r1= new Box<Integer>(5).m("anything",i->""+i);
16     new Box<String>("Hi").m(r1,s->s);
17   }
18 }
```

Example 7 marks the boundary of the family: with class generics, javac does flag the fault. javac rejects Example 7 at Line 7, with the error `Object cannot be safely cast to MyFoo`. The method `m` is an instance method of `Box<X>`, so `MyFoo` is an inner class of `Box<X>`, with an enclosing instance of type `Box<X>`. javac types `MyFoo` as `Box<X>.MyFoo`, as javac types a member class `MyFoo` of `Box`. So `MyFoo` is generic in `X`, and `other instanceof MyFoo` is the uncheckable type test `other instanceof Box<X>.MyFoo`.

The cast is flagged too. The following method `m` replaces the method `m` of Example 7. The type test against `String` replaces the type test against `MyFoo`, which javac rejects, and the cast of Example 2 follows it.

```
1  // Example 7 with a cast: the method m of Box
2    Object m(Object other, Function<X,String> f){
3      class MyFoo{ public X x(){ return x; } }
4      if(other instanceof String){ return new MyFoo(); }
5      MyFoo local= (MyFoo)other; // warning: [unchecked] unchecked cast
6      System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      return local;
8    }
```

javac accepts this program, but reports an unchecked warning for the cast at Line 5. At run time the cast succeeds, and the program fails as Example 2 fails. So with class generics the program fails only after an unchecked warning, as a program with the cast `(List<String>)o` may fail. In Examples 1 to 6 instead, javac reports nothing.

Thus for the generic parameters of the enclosing class, Java already exposes the captured generic types in the type of the local class, through the type of its enclosing instance. Java does not do the same for the generic parameters of the enclosing method: no type of the local class carries the instantiation of the generic parameters of a method.

3 The Impact of Generic Reification

With the new pile of examples, this should go much later in the paper. The "counter-example" is a perfect transition to jump into explaining funnelling and how it solves the issue, and reification would be a distracting tangent here. This should go near the end of the paper (and be shorter, and maybe be mentioned in the introduction that we explain how it's not about reification)

Not yet verified: the claims on Roslyn lifted classes, Rust (E0401, closure and opaque type identity, precise capturing), Swift, D, OCaml and SML local exceptions, and Scala outer tests.

A natural suspicion is that this problem is "simply" a consequence of generic erasure: a language that cannot distinguish the generic instantiations of a class at run time must have difficulties of this sort. There is some truth to this suspicion: javac accepts a cast that the JVM cannot check, because of erasure. However, javac does flag uncheckable type tests: for `o` of type `Object`, javac reports an unchecked warning for the cast `(List<String>)o`, and an error for `o instanceof List<String>`. javac reports nothing for Examples 1 to 6 because `MyFoo` is not generic: the type test of `other` against `MyFoo` has no generic argument to leave unchecked.

Reification does not change this by itself. A reified runtime can only check what the run-time identity of a class contains. If the run-time identity of `MyFoo` is just the declared class `MyFoo`, reification adds nothing to the cast `(MyFoo)other`. Java treats the class `MyFoo extends Foo<X>` as it treats the class `String extends Comparable<String>`: a cast inspects only the declared class. A cast to `String` does not inspect the generic argument of `Comparable<String>`, and a cast to `MyFoo` does not inspect the generic argument of `Foo<X>`. But the argument `String` of `Comparable<String>` is a constant of the class `String`: a reified runtime records it once, in the class table of `String`. The argument `X` of `Foo<X>` is not a constant of the class `MyFoo`: `X` is free in the declaration of `MyFoo`, and `X` changes between calls of `m`.

The implementations of method-local classes, and of the similar features of other languages, differ in the granularity of the identity of the local class. The identity of the local class matters in two places: in the type system, where it decides whether two types of the local class are the same type, and at run time, where it decides the result of casts and type tests. There are three granularities: (1) one identity per declared class, (2) one identity per instantiation of the enclosing scope, and (3) one identity per evaluation of the declaration.[2]

In (1), the declaration of a local class is a single point in the source code. So the local class `MyFoo` is one class, and the identity of `MyFoo` is its declaration point, the same for every instantiation of `m`. In (2), generic classes and generic methods are functions from generic arguments to concrete classes and methods: `static <X> Object m(..)` declares infinitely many methods `m<Integer>`, `m<String>`, and so on, one for each instantiation of `X`. In the same way, the generic class declaration `class Foo<X>` declares infinitely many classes `Foo<Integer>`, `Foo<String>`, and so on. So in (2) each method `m<T>` declares its own local class `MyFoo`, since the body of each `m<T>` is the body of a different method.

[2] Some languages avoid the choice by forbidding the capture of generic types. In Rust, a struct, function or impl declared inside a generic function can not use the generic parameters of the enclosing function (error E0401): the programmer must declare those generic parameters again on the inner item. Swift rejects a type declared inside a generic function.

cite Swift: type cannot be nested in generic function

Java checks casts and type tests with granularity (1): each class has a declaration point, and for each declaration point there is one `Class<?>` object, whose identity decides the result of the casts that the JVM checks. For class generics, the Java type system instead assumes granularity (2), as Example 7 shows. For member classes, the Java Language Specification states this granularity directly: if a generic class `C` has a non-generic member class `D`, then `C<String>.D` is a parameterized type, even though `D` is not generic [4, §4.5]. A local class declared in an instance method is an inner class of the enclosing class [4, §8.1.3], and javac gives the local class the same kind of type: in Example 7, `MyFoo` is `Box<X>.MyFoo`, so `Box<String>.MyFoo` and `Box<Integer>.MyFoo` are different types. A narrowing conversion from `Object` to such a parameterized type is unchecked [4, §5.1.6.2]: the cast `(MyFoo)other` causes an unchecked warning, and the type test `other instanceof MyFoo` is a compile-time error [4, §15.20.2]. These are the warning and the error of Example 7. However, generic methods are not treated in the same way: in Examples 1 to 6, the type of the local class is `MyFoo`, not `m<X>.MyFoo`. For method generics, the Java type system assumes granularity (1), as the casts do, so javac reports nothing and the program fails at run time. If the type of a local class contained the enclosing method and its generic instantiation, as in `Box<T1>.m<T2>.MyFoo`, then javac would flag the problem for generic methods too, as it does for generic classes.

A version of Java with reified generics could solve the mismatch in Example 7, between the granularity (1) of the casts and the granularity (2) of the types. But reification would not change Examples 1 to 6, unless the types of local classes take the form `A<T1>.m<T2>.MyFoo`. Moreover, Example 6 shows that the generic types needed by a local class can stay fully private: in Example 6, `X` occurs only in the private field `v` and in the body of `show`. Without an explicit way to put the needed generic types in the public API of the local class, such generic types stay private details. A type error caused by a fully private detail would make type checking non-modular. A failing dynamic cast caused by a fully private detail would make the private detail observable, and the language overall less predictable.

Granularity (2) is the default view for languages that implement parametric polymorphism by generative programming instead of generic programming [3], like C++ or early experiments with generic Java [1]. There each instantiation of `m` with different generic parameters is compiled separately, resulting in separate class declarations for each instantiation: generative programming forces (2) as the interpretation both at compile time and at run time. Each instantiation of `m` is a separate method of the compiled program, with its own local class `MyFoo`. Consider reinterpreting Example 1 in C++.

cite generative programming

```
1  // Example 1 in C++
2  class Object{ public: virtual ~Object(){} };
3  template<typename X> class Foo : public Object {
4      public: virtual X x() { return x(); } };
5  template<typename X>
6  Foo<X>* m(Object* other, const X* x) {
7      class MyFoo : public Foo<X> {
8          const X* v;
9        public: MyFoo(const X* v):v(v){} X x() { return *v; } };
10     MyFoo* d = dynamic_cast<MyFoo*>(other);
11     if(d == nullptr) { return new MyFoo(x); }
12     return d;
13 }
```

A C++ local class cannot use the parameters of its enclosing function, so `MyFoo` receives

[3] Also called templating or heterogeneous compilation.

`x` through its constructor. `Object` has a virtual destructor, since `dynamic_cast` requires a polymorphic source type. The calls `r1=m<int>(nullptr,&i)` and then `m<std::string>(r1,&s)` mirror Example 1. In the second call, the `dynamic_cast` at Line 10 returns `nullptr`: `m<int>` and `m<std::string>` declare two distinct classes `MyFoo`, and `r1` is an instance of the first one. Thus the second call returns a new `MyFoo`, as a sound type test should. In its diagnostics, `gcc` names the first class `m<int>(Object*, const int*)::MyFoo`: the instantiation of the method signature is the scope of the declaration of `MyFoo`. Inside the template `m<X>`, the C++ type system sees `MyFoo` as the dependent type `m<X>::MyFoo`. So C++ does expose `X` in the type of `MyFoo`: through the name of the enclosing scope instead of through a generic parameter of `MyFoo`. This is the form `m<T2>.MyFoo` that the Java type system does not have. D works in the same way, and its "Voldemort types" make the scope visible: a local struct `S` returned from the template function `f` has the type `f!int.S` when `f` is instantiated with `int`.

cite D Voldemort types

C# has no local classes, but its compiler lifts closures, local functions, iterators and async methods into generated classes. Inside a generic method `M<T>`, the generated classes are themselves generic in the captured `T`, as in `<>c__DisplayClass0_0<T>`. Since .NET reifies generics, the generated classes for `M<int>` and `M<string>` can be observed as distinct at run-time. A programmer can not name these generated classes in C# source, so a programmer can not write a cast to them.

cite Roslyn lowering

Rust closures and `impl Trait` opaque types capture the generic parameters of the enclosing function, and the type of the closure, or the opaque type, includes those generic parameters in its identity: the `TypeId` of a closure created in `m::<i32>` differs from the `TypeId` of the same closure created in `m::<String>` [9]. The precise capturing syntax of Rust 2024, `impl Trait + use<T>`, lists explicitly the generic parameters that an opaque type captures.

cite Rust precise capturing RFC 3617

Generative programming has subtle costs and wide-ranging consequences for language design, not just implementation, and complex performance consequences [7, 6]. However, these costs are not intrinsic to granularity (2): .NET keeps a single implementation for the instantiations at reference types and still gives each instantiation its own run-time identity [6].

Each of these sound designs follows granularity (2), and makes the captured generic types part of the identity of the local class: in the type system and at run time (C++, D, Rust), or only at run time (C# generated classes).

Overall, reification simply changes when a cast can be accepted by the type system. With reification we can have more expressive run time checks, so a more relaxed type system can still be sound.

Granularity (3) is a more extreme version of granularity (2): each evaluation of a declaration creates a new class, even for the same instantiation of the generic parameters. In OCaml, `let exception E of a in ...` creates a fresh exception constructor each time the declaration is evaluated; local exception declarations in Standard ML are generative in the same way. A handler for `E` does not catch an `E` raised by another evaluation of the declaration, whatever the instantiation of `a`. Scala has granularity (3) for captured values instead of captured types. An instance of an inner class `Inner` captures its enclosing instance `$outer`, and its type is the path-dependent type `a.Inner`. For the pattern `case i: a.Inner`, the Scala compiler emits an "outer test" that compares the captured `$outer` with `a`. Even here the type system exposes what is captured: inside the class of `a`, `Inner` means `this.Inner`. Granularity (3) is sound[4], but it is stricter than needed to avoid the problem at hand.

cite OCaml local exceptions, SML generative exceptions

cite Scala outer tests

[4] Granularity (3) also has many other useful applications, but those applications are unconnected with the problem at hand.

4 Funnelling

Granularities (1), (2) and (3) are points in a space of design options, but other points lie between them. Funnelling, the approach we present in this paper, lies between (1) and (2): in Example 1, `MyFoo` funnels `X` and becomes `MyFoo<X>`. For `other` of type `Object`, the existing rules of Java then flag `other instanceof MyFoo<X>` with an error and `(MyFoo<X>)other` with an unchecked warning, because `Object` does not determine `X`. These are the two compile-time points of the introduction. With reification, the same type test would be accepted by the compiler, and it would return false in the second call of `m`.

We now show funnelling on the core language of Section 5, a version of Featherweight Java [5] rich enough to express the problem. The core language has generics, type tests and local classes. A local class is written as an expression, like an anonymous class in Java, but with a name: `new MyFoo<Fs> extends Foo<Ts>(es){Ms}` declares the local class `MyFoo` and creates an instance of it. This brings method-local classes into an expression-based calculus, without modelling statements. The funnels `Fs` are the generic parameters of `MyFoo`: a funnel `X funnel` funnels the type variable `X` in scope. During reduction, `X funnel` becomes the ground type that instantiates `X`. The type test `if (e instanceof T x){ e1 } else { e2 }` binds `x` in `e1`; the core language has no casts. In the listings, we omit the empty `<>` of methods and calls, and we write `<X>` for `<X extends Object<>>`. This is Example 1 in the core language, without funnels:

```
class A<> extends Object<>{}
class B<> extends Object<>{ B<> onlyB(){ return new B<>(); } }
class Foo<X> extends Object<>{ X x(){ return this.x(); } }
class Broken<> extends Object<>{
  <X> Foo<X> m(Object<> other, X x){ return
    if (other instanceof MyFoo<> w){ w }
    else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
}
```

As in Java, the type of the local class is `MyFoo<>`, with no generic parameters. Section 5 presents the type rules and the small step reduction of the core language. The type rules of Section 5 encode funnelling. With those type rules, the example above is ill typed: the supertype `Foo<X>` of `MyFoo` mentions `X`, but `MyFoo` does not funnel `X`. Rule NEW-LOCAL-JAVA enables the unsound Java typing:

$$\text{NEW-LOCAL-JAVA}\quad \frac{\Delta \vdash C'\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \qquad \Delta;\Gamma \setminus \texttt{this}, C\texttt{<>}\ \texttt{this} \vdash Ms\ \text{Ok} \qquad \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<>}}$$

Rule NEW-LOCAL-JAVA uses the syntax `new C<> extends C'<Ts>(es){Ms}`. This syntax is an intermediate form between method-local classes and anonymous classes of Java. A method-local class `class C<> extends C'<Ts>{ C(){ super(es); } Ms }` is declared first, and then instantiated with `new C<>()`. An anonymous class `new C'<Ts>(es){Ms}` is declared and instantiated by a single expression. The syntax `new C<> extends C'<Ts>(es){Ms}` forms a (named) local class. A local class creates an instance as an anonymous class does, thus a local class is easy to use in an expression-based language. A local class also declares its own name, thus `instanceof` can test for a local class.

Rule NEW-LOCAL-JAVA types the methods Ms of the local class C under all the bounds Δ

and all the variables Γ in scope, except `this`, which has type C`<>`. The example above is well typed using the type rules of Section 5 and rule NEW-LOCAL-JAVA. The main expression is the first line of the reduction below, and it has type `B<>`. We now show the small step reduction using the abbreviations `eA`, `vA` and `vB`, and omitting the renaming of bound variables:

```
eA = new Broken<>().<A<>>m(new Foo<A<>>(), new A<>())
vA = new MyFoo<> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }
vB = new MyFoo<> extends Foo<B<>>(){ B<> x(){ return new B<>(); } }

   new Broken<>().<B<>>m(eA, new B<>()).x().onlyB()
→* new Broken<>().<B<>>m(vA, new B<>()).x().onlyB()
→* (if (vA instanceof MyFoo<> w){ w } else { vB }).x().onlyB()
→ vA.x().onlyB()
→ new A<>().onlyB()
```

The first call of `m` reduces `eA` to `vA` with rules SIMPLEADDSUPER, SIMPLEFOUND and IFELSE. The type of `vA` is `MyFoo<>`: the instantiation `A<>` of `X` shows only in the supertype `Foo<A<>>` and in the method `x`. In the second call of `m`, rule IFTHEN checks only that the class of `vA` is a subclass of `MyFoo`. Thus the second call returns `vA`, a `Foo<A<>>`, as a `Foo<B<>>`. The last expression calls `onlyB` on an `A<>`, and `A<>` does not have a method called `onlyB`. Rule SEARCHSUPER looks for `onlyB` up to `Object<>`, where no rule applies: the reduction is stuck.

With funnelling, `MyFoo` must funnel `X`, written `new MyFoo<X funnel>`, and instances of the local class are of type `MyFoo<X>`. Rule CAST-DOWN accepts a type test only when the static type determines `X`, as Java does. Thus `other instanceof MyFoo<X> w` is ill typed for `Object<> other`, and the method `m` is ill typed. This is the rule by which Java rejects `o instanceof List<String>` for `Object o`. If we change the signature of `m` as follows, the method `m` is well typed:

```
 <X> Foo<X> m(Foo<X> other, X x){ return
   if (other instanceof MyFoo<X> w){ w }
   else { new MyFoo<X funnel> extends Foo<X>(){ X x(){ return x; } } }; }
```

Now `eA` is a well typed expression of type `Foo<A<>>`, but the main expression is ill typed: the second call of `m` expects a `Foo<B<>>` as the first argument, but `eA` is a `Foo<A<>>`.

With reification, the funnelled method `m` with `Object<> other` would be well typed, but the reduction would change. In the second call of `m`, the type test of the `MyFoo<A<>>` created by the first call against `MyFoo<B<>>` is false. The second call returns a new `MyFoo<B<>>`, and the main expression reduces to `new B<>()`. Reification without funnelling does not help: if we allow the value `vA` to have type `MyFoo<>`, even a reified type test of `vA` against `MyFoo<>` is true.

5 A Core Language for Funnelling

We now present the core language of Section 4: its syntax, its small step reduction and its type rules. The type rules encode funnelling. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA gives the Java typing of local classes.

5.1 Syntax

Expressions	e	::=	$x \mid e.\texttt{<}Ts\texttt{>}m(es) \mid e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es) \mid e.f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$
			$\mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\texttt{\{}Ms\texttt{\}}$
			$\mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$
Types	T	::=	$C\texttt{<}Ts\texttt{>} \mid X$
Funnels	F	::=	$G \mid X\ \texttt{funnel}$
Ground Types	G	::=	$C\texttt{<}Gs\texttt{>}$
Type Env.	Γ	::=	$T_1\ x_1, \ldots, T_n\ x_n$
Bounds	Δ	::=	$X_1\ \texttt{extends}\ T_1, \ldots, X_n\ \texttt{extends}\ T_n$
Declaration	Dec	::=	$\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}$
(Field) Slots	S	::=	$T\ f\texttt{;}$
Methods	M	::=	$\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}}$
Class Decl.	K	::=	$\text{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$

An expression is a variable x, a method call $e.\texttt{<}Ts\texttt{>}m(es)$, a super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$, a field access $e.f$, an object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>(}es\texttt{)}$ for a top-level type, a local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$ or a type test $\texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$.[5] As in FJ, `this` is just a special case of a variable x. A method call $e.\texttt{<}Ts\texttt{>}m(es)$ calls the method m on the receiver e, with type arguments Ts and arguments es. A super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$ searches the method m starting from the class of the type T, instead of from the class of the receiver e. That is, a super call works as a flexible kind of super call, attempting to call the method m as if the receiver were of the supertype T. An object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>(}es\texttt{)}$ creates an instance of the top-level class $C\texttt{<}Ts\texttt{>}$. The arguments es are the values of the fields of C, followed by the arguments of the object creation of the superclass of C. A local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$ declares the local class C and creates an instance of C.

The funnels Fs serve a double role. From the perspective of the inside of the local class (the methods Ms), the funnels Fs declare the generics visible in the local class. From the perspective of the `new` expression itself, the funnels Fs instantiate those generic parameters, forming the actual type of the created instance.

To allow a local class instance to exist in the main expression, a funnel is either a ground type G or a funnelled type variable $X\ \texttt{funnel}$. In a method body, we expect all the funnels to be of the form $X\ \texttt{funnel}$. In the main expression, outside of the methods of local classes, we expect all the funnels to be of the form G. When a local class is nested in a method of a local class in the main expression, a mixture of funnels $X\ \texttt{funnel}$ and G can be present.

A local class has no fields: the arguments es are the arguments of the object creation of the superclass $C'\texttt{<}Ts\texttt{>}$. The methods Ms of a local class can use the variables and the type variables in scope. The type test $\texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$ reduces to e_1, with x bound to the value of e_0, when the value of e_0 is an instance of T; otherwise the type test reduces to e_2.

A type is a class type $C\texttt{<}Ts\texttt{>}$ or a type variable X. A ground type G is a type with no type variables. A ground type is the equivalent of a value, but at the type level. The conventional type environment Γ gives a type to each variable in scope. Bounds Δ give an upper bound to each type variable in scope. A declaration $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}$ declares the top-level class C with the type variables bounded by Δ, the superclass $C'\texttt{<}Ts\texttt{>}$, the

[5] The syntax `if (a instanceof B c)` is valid Java since Java 16.

methods Ms and the fields/slots Ss. A method `<`Δ`>`T_0 m`(`T_1 x_1`,` $\ldots$`,` T_n x_n`) { return` e`; }` has the type variables bounded by Δ, the return type T_0, the parameters $x_1, \ldots, x_n$ and the body e. Every method has a body; for simplicity, as in FJ, there are no abstract methods. A class declaration K is either a top-level declaration Dec or a local class. The superclass of every class, top-level or local, is a top-level class: local classes cannot be extended.

A program is a class table Decls and a main expression e. We require the following well-formedness conditions:

(A1) In Decls, class names are unique across top-level and local classes.
(A2) In a class, all the field names, including the field names of the superclasses, are disjoint.
(A3) In a class, top-level or local, all the method names are disjoint.
(A4) In a method, all the parameter names $x_1, \ldots, x_n$ are disjoint.
(A5) In a Δ, all the X are disjoint.
(A6) In a list of funnels Fs, all the funnelled type variables are disjoint.
(A7) There is no shadowing of X or x, except for `this`.
(A8) No parameter and no variable of an `if` is `this`.
(A9) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.

Well-formedness requirements A1-A9 are common requirements, such as uniqueness of names. To simplify scoping, no shadowing is allowed. A method body of a local class also binds `this`, thus shadowing the `this` of the enclosing method; this is the only form of shadowing.

5.2 Values and Contexts

$$
\begin{array}{rrcl}
\text{Values} & v & ::= & \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{)} \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}} \\
\text{Eval. Ctx.} & \mathcal{E} & ::= & \Box \mid \mathcal{E}\texttt{.<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
& & & \mid \mathcal{E}\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
& & & \mid \mathcal{E}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
& & & \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)\{}Ms\texttt{\}} \\
& & & \mid \texttt{if (}\mathcal{E}\ \texttt{instanceof}\ G\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
\text{Expr. Ctx.} & \mathcal{C} & ::= & \Box \mid \mathcal{C}\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
& & & \mid \mathcal{C}\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
& & & \mid \mathcal{C}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Fs\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
& & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)\{}Ms\texttt{\}} \\
& & & \mid \texttt{if (}\mathcal{C}\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
& & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ \mathcal{C}\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
& & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ \mathcal{C}\ \texttt{\}}
\end{array}
$$

5.3 Declarations

Define K declared
Dec declared iff Dec $\in$ Decls

`new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` declared
iff `new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` occurs in e
where K declared
`<_>_` m`(_) { return` e`; }` $\in K$

Define $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts'$

$\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$
where `class` C`<`Δ`>` `extends` `_<_>` `{ _ _ }` declared

$\mathsf{scope}(C_0\texttt{<}Ts\texttt{>}) = Fs \overset{\Delta,\Delta'}{\Longleftrightarrow} Ts$
where `<`Δ'`>_` m`(_) { return` e`; }` $\in C$
`new` C_0`<`Fs`>` `extends` `_<_>(_){_}` occurs in e
$\mathsf{scope}(\mathrm{generic}(C)) = \Delta; _$

Define $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts'$ $\quad (\Delta; Ts) \cdot (\Delta'; Ts') = \Delta''; Ts''$

$\emptyset \overset{\Delta}{\Longleftrightarrow} \emptyset = \emptyset; \emptyset$

X `funnel`$, Fs \overset{\Delta}{\Longleftrightarrow} T, Ts = (X$ `extends` $\Delta(X); T) \cdot (Fs \overset{\Delta}{\Longleftrightarrow} Ts)$

$G, Fs \overset{\Delta}{\Longleftrightarrow} G, Ts = Fs \overset{\Delta}{\Longleftrightarrow} Ts$

$(\Delta; Ts) \cdot (\Delta'; Ts') = \Delta, \Delta'; Ts, Ts'$

Define $M \in K$ $\quad M \in C$ $\quad \mathrm{generic}(C) = T$ $\quad |F| = T$

$M \in K$ iff $M \in Ms$
where $K \in \{$`class _<_> extends _<_> { `Ms` _ }`$, $`new _<_> extends _<_>(_){`$Ms$`}}`

$M \in C$ iff $M \in K$
where K declared
$K \in \{$`class` C`<_> extends _<_> { _ _ }`$, $`new` C`<_> extends _<_>(_){_}}`

$\mathrm{generic}(C) = C$`<`$\mathrm{dom}(\Delta)$`>`
where `class` C`<`Δ`>` `extends _<_> { _ _ }` declared

$\mathrm{generic}(C) = C$`<`$|Fs|$`>`
where `new` C`<`Fs`>` `extends _<_>(_){_}` declared

$|X$ `funnel`$| = X$

$|G| = G$

By well-formedness requirement A1, a class name C identifies at most one K with K declared. $M \in K$ holds when M is a method of the class declaration K, top-level or local. $M \in C$ holds when M is a method of the declared class C, top-level or local. For a class type C`<`Ts`>`, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0$ gives the bounds Δ_0 of the type variables that are in scope for all the methods of C.[6] It also gives the types Ts_0 that the arguments Ts assign to the type variables in Δ_0. For a top-level class, Δ_0 is the bounds of the class and $Ts_0 = Ts$. $\mathsf{scope}(_) = \Delta; Ts$ is expected to always return a Δ and a Ts of the same length. This is enforced at the usage points, where we use the notation $T[\Delta = Ts]$, the simultaneous substitution, defined only when Δ and Ts have the same length.

For a type variable X, $\Delta(X)$ is the bound of X in Δ. $\mathrm{dom}(\Delta)$ is the sequence of the type variables in Δ, and $\mathrm{cod}(\Delta)$ is the sequence of the bounds in Δ. $\mathsf{FTV}(T)$ is the set of the type variables occurring in T; for a sequence, FTV is the union over its elements. The figures leave the following conventions implicit. A sequence $e_1, \ldots, e_n$, or es, separates its elements with commas; a comma also concatenates two sequences, and either sequence may be empty. A function or a judgement applied to sequences stands for one application at every position, and requires the sequences to have the same length: $|F_1, \ldots, F_n|$ is $|F_1|, \ldots, |F_n|$; $\Delta \vdash T_1, \ldots, T_n$ Ok is $\Delta \vdash T_1$ Ok $\quad \ldots \quad \Delta \vdash T_n$ Ok; $\Delta \vdash T_1, \ldots, T_n <: T'_1, \ldots, T'_n$ is $\Delta \vdash T_1 <: T'_1 \quad \ldots \quad \Delta \vdash T_n <: T'_n$; $\Delta; \Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n$ is $\Delta; \Gamma \vdash e_1 : T_1 \quad \ldots \quad \Delta; \Gamma \vdash e_n : T_n$. Finally, $m \notin Ms$ holds when no method in Ms has name m, and $\mathrm{dom}(Ms)$ is the set of the names of the methods in Ms. $\Gamma \setminus x$ is Γ without the variable x. For a set of type variables Xs, $\Gamma|_{Xs}$ keeps the variables $T\ x$ of Γ such that $\mathsf{FTV}(T) \subseteq Xs$.

[6] Note that the type variables declared by each individual generic method are not accounted for here.

The pairing $Fs \overset{\Delta}{\Longleftrightarrow} Ts$ goes through the funnels Fs and the arguments Ts together, one position at a time, where $\emptyset$ is the empty sequence. At the position of a funnel X `funnel`, it adds X `extends` $\Delta(X)$ to the bounds and the argument to the types. At the position of a ground funnel G, the argument must be G itself, and nothing is added. The operator $(\Delta; Ts) \cdot (\Delta'; Ts')$ concatenates two results of a pairing, component by component. An expression e' occurs in e when $e = \mathcal{C}[e']$, for an expression context $\mathcal{C}$. Expression contexts $\mathcal{C}$ do not enter the methods of a local class; thus $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ has at most one value.

5.4 Operational Semantics

We now present the reduction $e \to e'$ of the main expression. The reduction is small step, and uses the values v and the evaluation contexts $\mathcal{E}$.

$$\boxed{e \to e'}$$

simpleFieldFound

$$\frac{\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f_i \in f_1, \ldots, f_n}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1, \ldots, v_n, vs\texttt{)}\,.\,f_i \to v_i}$$

simpleFieldSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f \notin f_1, \ldots, f_n \qquad C \neq \texttt{Object}}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1, \ldots, v_n, vs\texttt{)}\,.\,f \to \texttt{new}\ C'\texttt{<}Ts[\Delta = Gs]\texttt{>(}vs\texttt{)}\,.\,f}$$

innerFieldSuper

$$\frac{}{\texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}}\,.\,f \to \texttt{new}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)}\,.\,f}$$

ifThen

$$\frac{v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\} \qquad \emptyset \vdash C\texttt{<}Gs\texttt{>} <: C'\texttt{<_>} \qquad \mathsf{fresh}(e_1) = e_1'}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_1'[x = v]}$$

ifElse

$$\frac{v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\} \qquad \emptyset \not\vdash C\texttt{<}Gs\texttt{>} <: C'\texttt{<_>}}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_2}$$

$$\textsc{simpleAddSuper}\ \frac{v = \texttt{new}\ C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m\texttt{(}vs\texttt{)} \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)}}$$

$$\textsc{simpleFound}\ \frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}}\ \mathsf{declared} \\ M \in Ms \qquad \mathsf{fresh}(M) = \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \texttt{\{}\ \texttt{return}\ e_0\texttt{;}\ \texttt{\}} \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}v_1, \ldots, v_n\texttt{)} \to e}$$

$$\textsc{innerFound}\ \frac{\begin{array}{c}v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}} \\ M \in Ms \qquad \mathsf{fresh}(M) = \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \texttt{\{}\ \texttt{return}\ e_0\texttt{;}\ \texttt{\}} \\ e = e_0[\Delta' = Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.<}Gs'\texttt{>}m\texttt{(}v_1, \ldots, v_n\texttt{)} \to e}$$

$$\textsc{innerSuper}\ \frac{v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>(_)\{}Ms\texttt{\}} \qquad m \notin Ms}{v_0\texttt{.<}Gs'\texttt{>}m\texttt{(}v_1, \ldots, v_n\texttt{)} \to v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}v_1, \ldots, v_n\texttt{)}}$$

$$\textsc{searchSuper}\ \frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}}\ \mathsf{declared} \qquad m \notin Ms \qquad C \neq \texttt{Object}}{v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)} \to v\texttt{.[}C'\texttt{<}Ts[\Delta = Gs]\texttt{>]<}Gs'\texttt{>}m\texttt{(}vs\texttt{)}} \qquad \textsc{ctx}\ \frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

Most of the rules are standard. Rules simpleFieldFound, simpleFieldSuper and innerFieldSuper read a field. Rule simpleFieldFound reads a field declared by the class of the object. Rule simpleFieldSuper reads a field not declared by the class C of the object from the object of the superclass of C, made of the remaining values. A local class declares no fields, so rule innerFieldSuper reads every field from the object of the superclass.

Rules ifThen and ifElse test the class of the value v against the class C' of the type test, ignoring the type arguments of C'. Rule ifThen reduces to e_1 with x replaced by v; rule ifElse reduces to e_2.

Rules simpleAddSuper, simpleFound, innerFound, innerSuper and searchSuper call a method. Rule simpleAddSuper turns a call on an object of a top-level class into a super call starting from the class of the object. Rule simpleFound runs the method m found in the class of a super call, replacing the type variables of the class and of the method, `this` and the parameters. Rule innerFound runs the method m found in a value of a local class; the funnelled type variables are already replaced in that value, so only the type variables of the method are replaced. Rule innerSuper turns a call on a value of a local class without the method m into a super call starting from its superclass. Rule searchSuper moves a super call to the superclass when the class does not declare m. Rule ctx reduces inside an evaluation context.

Rules ifThen, simpleFound and innerFound also use the relation fresh. The names bound in an expression e are the variable of every `if` in e and the type variables and parameters of every method of a local class in e. The names bound in a method M are the type variables of its signature, its parameters, and the names bound in its body. An expression $\mathsf{fresh}(e)$ is e where every name bound in e is renamed into a fresh name, that is a name occurring nowhere in the program; distinct binders get distinct fresh names. A method $\mathsf{fresh}(M)$ is M renamed in the same way. Thus fresh is not a function: a rule using fresh can choose any fresh names. The variable `this` is never renamed. Renaming is needed because a

value of a local class carries the code of its methods. For example, consider a local class `L` whose method `m` has a parameter `x` and calls `m` on a captured value `y` of class `L`:

```
Object<> k(L<> y){ return
  new L<> extends Object<>(){ Object<> m(Object<> x){ return y.m(x); } }; }
```

In a call of `k` with a value `v` of class `L`, the body of `m` becomes `v.m(x)`, and `v` contains a copy of the method `m` binding `x` again. Without renaming, rule innerFound would replace `x` also in the method `m` of `v`. With renaming, requirement A7 holds for the main expression at every step of the reduction. Rules ifThen, simpleFound and innerFound place values under the binders of the expression they substitute into. Those binders are fresh, so the values do not bind the same names. A substitution can copy a value, so several copies of a value can bind the same names; those copies are never nested, so they do not shadow each other. Thus a substitution never reaches a type variable or a variable bound inside the substituted expression.

5.5 Typing

$$\boxed{\Delta;\Gamma \vdash e : T}$$

SUBS
$$\frac{\Delta;\Gamma \vdash e : T_1 \qquad \Delta \vdash T_2 \text{ Ok} \qquad \Delta \vdash T_1 <: T_2}{\Delta;\Gamma \vdash e : T_2}$$

VAR
$$\frac{}{\Delta;\Gamma \vdash x : \Gamma(x)}$$

NEW-OBJECT
$$\frac{}{\Delta;\Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

DECLFIELD
$$\frac{\Delta;\Gamma \vdash e : C\texttt{<}Ts'\texttt{>} \qquad \texttt{class } C\texttt{<}\Delta'\texttt{> extends _<_> \{ _ } Ss \texttt{ \} declared} \qquad T\ f\texttt{;} \in Ss}{\Delta;\Gamma \vdash e.f : T[\Delta' = Ts']}$$

CALL
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts_0\texttt{>} \\ \text{generic}(C_0) = C_0\texttt{<}Ts_2\texttt{>} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in C_0 \\ \forall i \in 0..n.\, T_i' = T_i[Ts_2, \Delta_1 = Ts_0, Ts_1] \\ \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \text{cod}(\Delta_1)[Ts_2, \Delta_1 = Ts_0, Ts_1] \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

CALL-LB-RUNTIME
$$\frac{\begin{array}{c}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C_0\texttt{<}Ts_0\texttt{>} \\ \texttt{class } C_0\texttt{<}\Delta_0\texttt{> extends _<_> \{ }Ms\texttt{ _ \}} \text{ declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in Ms \\ \forall i \in 0..n.\, T_i' = T_i[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \texttt{class } C\texttt{<_> extends _<_> \{ _ _ \}} \text{ declared} \qquad \Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok} \\ \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \text{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \Delta;\Gamma \vdash e_0 : C\texttt{<}Ts\texttt{>} \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.[}C\texttt{<}Ts\texttt{>]<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

NEW-TOP

$$\frac{\texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ T_1\ f_1\texttt{;}\ \ldots\ T_n\ f_n\texttt{;}\ \}\ \text{declared} \qquad \Delta \vdash C\texttt{<}Ts'\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash e_1,\ldots,e_n : T_1,\ldots,T_n[\Delta' = Ts'] \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts[\Delta' = Ts']\texttt{>(}es\texttt{)} : _}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Ts'\texttt{>(}e_1,\ldots,e_n,es\texttt{)} : C\texttt{<}Ts'\texttt{>}}$$

NEW-LOCAL

$$\frac{\begin{array}{c}\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}\ \text{consistent} \\ Fs \overset{\Delta}{\Longleftrightarrow} |Fs| = \Delta';_ \qquad \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \\ \Delta' \vdash C\texttt{<}|Fs|\texttt{>}\ \text{Ok} \qquad \Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \\ \Delta';\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}, C\texttt{<}|Fs|\texttt{>}\ \texttt{this} \vdash Ms\ \text{Ok} \\ \mathsf{overrideOk}(\Delta', C_0\texttt{<}Ts\texttt{>}, Ms)\end{array}}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<}|Fs|\texttt{>}}$$

IF-INSTANCEOF

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T\ \text{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \\ \Delta;\Gamma, T\ x \vdash e_1 : T' \qquad \Delta;\Gamma \vdash e_2 : T'\end{array}}{\Delta;\Gamma \vdash \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} : T'}$$

513 **Define** K consistent
$\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$ consistent
iff $Ts = Ts'[Fs' = |Fs|]$
514 $\mathrm{dom}(Ms') = \mathrm{dom}(Ms)$
$\mathsf{overrideOk}(C\texttt{<}|Fs|\texttt{>}, Ms)$
where $\texttt{new}\ C\texttt{<}Fs'\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts'\texttt{>(_)\{}Ms'\texttt{\}}$ declared

515 The rule NEW-LOCAL is the heart of funnelling, including subtle management of type
516 environments. The rule constructs a restricted set of type bounds and variable bindings
517 reflecting only the explicitly-funnelled type variables: Δ' includes only the explicitly-funnelled
518 type variables, and $\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}$ keeps only the variables of Γ, other than `this`, whose
519 types mention only type variables of the result type $C\texttt{<}|Fs|\texttt{>}$ (which must be well-kinded
520 under Δ'). The bounds Δ' are computed by $Fs \overset{\Delta}{\Longleftrightarrow} |Fs|$, which is undefined when a funnelled
521 type variable is not in Δ. However, the rule still uses the *full* ambient environments for
522 checking certain subexpressions. This is because some $e \in es$ may initially contain uses
523 of *non*-funnelled type variables (e.g., an explicit call), but will reduce to a value which no
524 longer depends on such type variables. This is why the premise $\Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _$
525 checks the effective supertype instance under $\Delta;\Gamma$, but the premise $\Delta' \vdash C_0\texttt{<}Ts\texttt{>}$ Ok ensures
526 the resulting type of that expression satisfies the funnelling constraints by checking kinding
527 under the more restrictive Δ'.

528 The local class K in K consistent agrees with its declaration, found by the *where* line of
529 the definition. The supertype and the method signatures of the local class are those of the
530 declaration, with the funnelled type variables of Fs' replaced by $|Fs|$. The local class and
531 the declaration have the same method names. For a local class in Decls, the declaration is
532 the local class itself; thus every local class in Decls is consistent. In the main expression, a
533 local class is a copy of its declaration where reduction has replaced type variables with types
534 and variables with values. Rules CALL and SUB-INHERIT-LOCAL read the declaration, while
535 the reduction runs the methods of the local class itself.

5.6 Typing of Programs, Classes and Methods

$\boxed{\vdash \mathsf{Decls}\ e\ \mathsf{Ok}}$

PROG-OK
$$\frac{\forall \mathsf{Dec} \in \mathsf{Decls}.\ \mathsf{Dec}\ \mathsf{Ok} \qquad \emptyset;\emptyset \vdash e : _}{\vdash \mathsf{Decls}\ e\ \mathsf{Ok}}$$

$\boxed{\Delta;\Gamma \vdash M\ \mathsf{Ok}}$

METHOD-OK
$$\frac{\Delta,\Delta' \vdash \mathrm{cod}(\Delta')\ \mathsf{Ok} \qquad \Delta,\Delta' \vdash T_0,\ldots,T_n\ \mathsf{Ok} \qquad \Delta,\Delta';\Gamma,T_1\ x_1,\ldots,T_n\ x_n \vdash e : T_0}{\Delta;\Gamma \vdash \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1,\ldots,T_n\ x_n\texttt{)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}}\ \mathsf{Ok}}$$

$\boxed{\mathsf{Dec}\ \mathsf{Ok}}$

CLASS-OK
$$\frac{\begin{array}{c}\texttt{class}\ C'\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ \texttt{_}\ \texttt{_}\ \texttt{\}}\ \mathsf{declared} \qquad \Delta \vdash \mathrm{cod}(\Delta)\ \mathsf{Ok} \\ \Delta \vdash C'\texttt{<}Ts\texttt{>}\ \mathsf{Ok} \qquad \forall T\ f\texttt{;} \in Ss.\ \Delta \vdash T\ \mathsf{Ok} \\ \Delta;C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}\ \texttt{this} \vdash Ms\ \mathsf{Ok} \\ \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)\end{array}}{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}\ \mathsf{Ok}}$$

Define $\mathsf{overrideOk}(\Delta, T, Ms)$ $\mathsf{overrideOk}(T, M)$

$\mathsf{overrideOk}(\Delta, T, Ms)$ iff $\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, Ms)$ for every $C\texttt{<}Ts\texttt{>}$ such that $\Delta \vdash T <: C\texttt{<}Ts\texttt{>}$

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<_>_}\ m\texttt{(_)}\ \texttt{\{}\ \texttt{_}\ \texttt{\}})$ iff $m \notin Ms$
where $\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}}$ declared

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1,\ldots,T_n'\ x_n\texttt{)}\ \texttt{\{}\ \texttt{_}\ \texttt{\}})$
iff $\mathrm{cod}(\Delta'')[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0'',\ldots,T_n''[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0',\ldots,T_n'$
where $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ Ms\ \texttt{_}\ \texttt{\}}$ declared
$\texttt{<}\Delta''\texttt{>}T_0''\ m\texttt{(}T_1''\ x_1',\ldots,T_n''\ x_n'\texttt{)}\ \texttt{\{}\ \texttt{_}\ \texttt{\}} \in Ms$

$\mathsf{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1,\ldots,T_n'\ x_n\texttt{)}\ \texttt{\{}\ \texttt{_}\ \texttt{\}})$
iff $\mathrm{cod}(\Delta'')[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
$T_0'',\ldots,T_n''[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0',\ldots,T_n'$
where $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}}$ declared
$\texttt{<}\Delta''\texttt{>}T_0''\ m\texttt{(}T_1''\ x_1',\ldots,T_n''\ x_n'\texttt{)}\ \texttt{\{}\ \texttt{_}\ \texttt{\}} \in Ms$

Rule CLASS-OK types the methods of a top-level class with `this` of type $C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}$. A method that a class declares again must keep the signature it has in every superclass, up to the names of its own type variables: CALL reads the signature from the class chosen by subsumption, while the reduction runs the method of the class of the receiver. The last case of $\mathsf{overrideOk}(T, M)$ compares a method of a local class with the method of the same name in the declaration of the local class, in the same way.

5.7 Kinding, Subtyping and Checked Casts

$$\boxed{\Delta \vdash T \text{ Ok}}$$

$$\text{WF-VAR}\ \frac{X\ \texttt{extends}\ _ \in \Delta}{\Delta \vdash X\ \text{Ok}} \qquad \text{WF-CLASS}\ \frac{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0 \qquad \Delta \vdash Ts\ \text{Ok} \qquad \Delta \vdash Ts_0 <: \mathrm{cod}(\Delta_0)[\Delta_0 = Ts_0]}{\Delta \vdash C\texttt{<}Ts\texttt{>}\ \text{Ok}}$$

$$\boxed{\Delta \vdash T <: T'}$$

$$\text{SUB-REFL}\ \frac{}{\Delta \vdash T <: T} \qquad \text{SUB-INHERIT}\ \frac{\texttt{class}\ C\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ _\ _\ \texttt{\}}\ \text{declared} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}[\Delta_0 = Ts_0] <: T'}{\Delta \vdash C\texttt{<}Ts_0\texttt{>} <: T'}$$

$$\text{SUB-INHERIT-LOCAL}\ \frac{\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>(_)\{_\}}\ \text{declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T'}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T'} \qquad \text{SUB-VAR}\ \frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

$$\boxed{\Delta \vdash T \rightsquigarrow T'}$$

$$\text{CAST-UP}\ \frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'} \qquad \text{CAST-VAR}\ \frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

$$\text{CAST-DOWN}\ \frac{\mathrm{generic}(C) = C\texttt{<}Ts''\texttt{>} \qquad \emptyset \vdash C\texttt{<}Ts''\texttt{>} <: C'\texttt{<}Ts'\texttt{>} \qquad \mathsf{FTV}(Ts'') \subseteq \mathsf{FTV}(Ts') \qquad \Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>}}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

Rule WF-CLASS kinds both top-level and local classes, including a local class written in a method of another local class. Every type in Ts_0 is checked against the bound of its type variable in Δ_0. A funnelled type variable that is not in scope where its local class C is written leaves $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ undefined; thus no type $C\texttt{<}Ts\texttt{>}$ is well kinded.

5.8 Substitution of Type Variables

Define $e[Xs = Ts]$ $M[Xs = Ts]$ $F[Xs = Ts]$ $\Delta[Xs = Ts]$ $T[Xs = Ts]$

$$
\begin{array}{lcl}
x[Xs = Ts] & = & x \\
e.\texttt{<}Ts'\texttt{>}m\texttt{(}es'\texttt{)}[Xs = Ts] & = & e[Xs = Ts].\texttt{<}Ts'[Xs = Ts]\texttt{>}m\texttt{(}es'[Xs = Ts]\texttt{)} \\
e.\texttt{[}T'\texttt{]<}Ts''\texttt{>}m\texttt{(}es'\texttt{)}[Xs = Ts] & = & e[Xs = Ts].\texttt{[}T'[Xs = Ts]\texttt{]<}Ts''[Xs = Ts]\texttt{>} \\
 & & m\texttt{(}es'[Xs = Ts]\texttt{)} \\
e.f[Xs = Ts] & = & e[Xs = Ts].f \\
\texttt{new } C\texttt{<}Ts'\texttt{>(}es\texttt{)}[Xs = Ts] & = & \texttt{new } C\texttt{<}Ts'[Xs = Ts]\texttt{>(}es[Xs = Ts]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof }T'\ x\texttt{)} & = & \texttt{if (}e_0[Xs = Ts]\texttt{ instanceof }T'[Xs = Ts]\ x\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [Xs = Ts] & & \texttt{\{ }e_1[Xs = Ts]\texttt{ \} else \{ }e_2[Xs = Ts]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts'\texttt{>} & = & \texttt{new } C\texttt{<}Fs[Xs = Ts]\texttt{> extends } C'\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [Xs = Ts] & & \texttt{(}es[Xs = Ts]\texttt{)\{}Ms[Xs = Ts]\texttt{\}} \\
X'\texttt{ funnel}[Xs = Ts] & = & X'\texttt{ funnel} \quad \text{if } X' \notin Xs \\
X_i\texttt{ funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i \\
G[Xs = Ts] & = & G \\
\texttt{<}\Delta\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_k'\ x_k\texttt{)} & = & \texttt{<}\Delta[Xs = Ts]\texttt{>}T_0'[Xs = Ts]\ m\texttt{(} \\
\texttt{\{ return }e\texttt{; \}}\ [Xs = Ts] & & \quad T_1'[Xs = Ts]\ x_1, \ldots, T_k'[Xs = Ts]\ x_k\texttt{)} \\
 & & \texttt{\{ return }e[Xs = Ts]\texttt{; \}} \\
(X'\texttt{ extends }T')[Xs = Ts] & = & X'\texttt{ extends }T'[Xs = Ts] \\
C\texttt{<}Ts'\texttt{>}[Xs = Ts] & = & C\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
X'[Xs = Ts] & = & X' \quad \text{if } X' \notin Xs \\
X_i[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i
\end{array}
$$

Substitution replaces every occurrence of a type variable, *including* the ones in the method signatures of a method-local class. Substitution does not rename the type variables it meets. No renaming is needed: the substituted types are ground, and the domain of the substitution never contains a type variable bound inside the substituted term, by requirement A7 on the main expression. In the domain of a substitution, bounds Δ stand for $\mathrm{dom}(\Delta)$, and funnels Fs stand for their funnelled type variables: $T[F_1, \ldots, F_n = T_1, \ldots, T_n]$ replaces X_i with T_i for every $i \in 1..n$ such that $F_i = X_i$ `funnel`; the ground funnels are ignored. In the same way, a sequence of types $T_1', \ldots, T_n'$ in the domain stands for its type variables: $T[T_1', \ldots, T_n' = T_1, \ldots, T_n]$ replaces X_i with T_i for every $i \in 1..n$ such that $T_i' = X_i$; the ground types are ignored. Thus for $\mathrm{generic}(C) = C\texttt{<}Ts_0\texttt{>}$, the substitution $T[Ts_0 = Ts]$ replaces the type variables of the class C, top-level or local. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground. Thus $X_i\texttt{ funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] = T_i$ is a ground type G, and the result is again a funnel.

5.9 Substitution of Variables

Define $e[xs = vs]$ $M[xs = vs]$

$$
\begin{array}{lcl}
x[xs = vs] & = & x \quad \text{if } x \notin xs \\
x_i[x_1, \ldots, x_n = v_1, \ldots, v_n] & = & v_i \\
e.\texttt{<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{<}Ts\texttt{>}m(es'[xs = vs]) \\
e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{[}T\texttt{]<}Ts\texttt{>}m(es'[xs = vs]) \\
e.f[xs = vs] & = & e[xs = vs].f \\
\texttt{new } C\texttt{<}Ts\texttt{>(}es\texttt{)}[xs = vs] & = & \texttt{new } C\texttt{<}Ts\texttt{>(}es[xs = vs]\texttt{)} \\
\texttt{if (}e_0 \texttt{ instanceof } T\ x'\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [xs = vs] & = & \texttt{if (}e_0[xs = vs] \texttt{ instanceof } T\ x'\texttt{)} \\
 & & \texttt{\{ }e_1[xs = vs]\texttt{ \} else \{ }e_2[xs = vs]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs = vs] & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
 & & \texttt{(}es[xs = vs]\texttt{)\{}Ms[xs = vs]\texttt{\}} \\
 & & \quad \text{if } \texttt{this} \notin xs \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs, \texttt{this}, xs' = vs, v, vs'] & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
 & & \texttt{(}es[xs, \texttt{this}, xs' = vs, v, vs']\texttt{)\{}Ms[xs, xs' = vs, vs']\texttt{\}} \\
\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x'_1, \ldots, T_k\ x'_k) \\
\texttt{\{ return } e\texttt{; \}}\ [xs = vs] & = & \texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x'_1, \ldots, T_k\ x'_k) \\
 & & \texttt{\{ return } e[xs = vs]\texttt{; \}}
\end{array}
$$

A method-local class binds `this`, as in Java: inside the methods of a method-local class, `this` is the local object, and the outer `this` is hidden. Thus substitution of variables enters the methods of a method-local class for every variable except `this`. The variable x' of an `if` and the parameters $x'_1, \ldots, x'_k$ of a method are never in xs, by requirement A7 on the main expression; so no renaming is needed.

...

5.10 Java Typing of Local Classes

Rule NEW-LOCAL-JAVA types local classes as Java does. A local class has no funnels, and its methods are typed under the full Δ and Γ in scope, except for `this`. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA makes our system unsound for the same reason Java is unsound. Consider the method `mk` below, the method `m` of Example 1 in our syntax:

```
<X> Foo<X> mk(Object<> other, X x){ return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
```

Rule NEW-LOCAL-JAVA gives to the object created by `mk` the type `MyFoo<>`. Rule SUB-INHERIT-LOCAL reads the declaration of `MyFoo`, so `MyFoo<>` is a subtype of `Foo<X>`, where `X` is the type variable of `mk`. The type `MyFoo<>` does not say which type `X` stood for when the object was created. Since generic(`MyFoo`) = `MyFoo<>` has no type variables, rule CAST-DOWN accepts the type test `other instanceof MyFoo<>` from `Object<>`. Thus the call `a.<String>mk(a.<Integer>mk(o, i), s)` has type `Foo<String>`, but it returns a `Foo<Integer>`. This is the unsoundness of Java. It is there even if we keep the restriction of Java that the name of a local class can only be written inside the method declaring it, as `mk` does. Our formal system shows two more failure points. They are the same failure, shown more explicitly:

- Reduction does not preserve types. The call `a.<Integer>mk(o, i)` has type `Foo<Integer>`. The value `o` is not a `MyFoo`, so the type test fails and the call takes the `else` branch. It reduces to the value

```
new MyFoo<> extends Foo<Integer>(){ Integer x(){ return i; } }
```

The type of this value is `MyFoo<>`, and `MyFoo<>` is not a subtype of `Foo<Integer>`. The supertypes of `MyFoo<>` come from the declaration of `MyFoo` in `mk`, not from the value: rule SUB-INHERIT-LOCAL derives $\emptyset \vdash$ `MyFoo<>` $<:$ `Foo<X>`, where `X` is the type variable of `mk`. Subtyping does not check kinding, so this judgement holds in the empty bounds $\emptyset$, where `X` is not in scope. However, `Foo<X>` is not well kinded under $\emptyset$, so rule SUBS cannot give the value the type `Foo<X>` either. Java has no typing of values during reduction, so in Java this failure is invisible.

Typing depends on the names of type variables. Our system lets the name of a local class be written anywhere. Consider the methods

```
<Y> MyFoo2<> mk2(Y y){
  return new MyFoo2<> extends Foo<Y>(){ Y x(){ return y; } }; }
<Y> Foo<Y> leak(MyFoo2<> z){ return z; }
```

The method `leak` is well typed, since the type variable `Y` of `leak` has the same name as the type variable `Y` of `mk2`. Renaming `Y` into `Z` in `leak` makes `leak` ill typed. The call

```
a.<String>leak(a.<Integer>mk2(i))
```

has type `Foo<String>`, but it returns a `Foo<Integer>`, without type tests.

Funnelling solves the problem in all three forms. Under rule NEW-LOCAL, the object created by `mk` has type `MyFoo<X>`, written `new MyFoo<X funnel> extends Foo<X>(){..}`. Since generic(`MyFoo`) = `MyFoo<X>`, rule CAST-DOWN accepts the type test `instanceof MyFoo<X>` from `Foo<X>`, but not from `Object<>`. Thus the funnelled `mk` takes `Foo<X> other` instead of `Object<> other`, and tests `other instanceof MyFoo<X> w`. The call `a.<Integer>mk(o, i)`, with `o` of type `Foo<Integer>`, reduces to a value of type `MyFoo<Integer>`, a subtype of `Foo<Integer>`.

...

briefly note that Java allows extra, "fresh" generic parameters in local classes

call out that we're allowed to declare return types that mention an inner class; inner classes are "public", and our approach works even in this case, the more restrictive case of Java is naturally covered

articulate early: It's not just about generic reification: just checking type parameters aren't enough, as our MyFoo cases have no generic params. You need to *deeply* check reified generics, you need to check not just the base type but also check the choice(s) of parent types from the context into the runtime check.

6 (Un)Soundness of Method Local Classes

With our language primitives in place, we now consider type safety of two variants of our language. The two variants differ only in the typing of local classes:

1. ζ(Java) types local classes with rule NEW-LOCAL-JAVA, as Java does;
2. ζ(funnelled) types local classes with rule NEW-LOCAL.

Both variants use the type test of Java: rule IF-INSTANCEOF accepts only checked casts, and at run time rules IFTHEN and IFELSE compare class names only. As shown in Section 5, ζ(Java) is unsound. We now show that ζ(funnelled) is sound. In the statements below, the program Decls e satisfies well-formedness requirements A1-A9, and Dec Ok holds for every Dec $\in$ Decls.

▶ **Lemma 1** (Well-Formedness). *If $e \to e'$, then the program* Decls e' *satisfies well-formedness requirements A1-A9.*

▶ **Theorem 2** (Progress). *If* $\emptyset;\emptyset \vdash e : T$*, then either* e *is a value* v*, or there is an* e' *such that* $e \to e'$*.*

▶ **Theorem 3** (Preservation). *If* $\emptyset;\emptyset \vdash e : T$ *and* $e \to e'$*, then* $\emptyset;\emptyset \vdash e' : T$*.*

▶ **Corollary 4** (Type Soundness). *If* $\vdash$ Decls e_0 Ok *with* $\emptyset;\emptyset \vdash e_0 : T$*, and* $e_0 \to^* e$ *with no* e' *such that* $e \to e'$*, then* e *is a value* v *and* $\emptyset;\emptyset \vdash v : T$*.*

7 Related Work

It is not uncommon to have "intentional unsoundness" in industrial languages, such as Java's covariant array subtyping, or TypeScript's covariant method subtyping. However these are intentionally designed into the language to improve ease of use, and in most cases (like Java's array handling) problematic uses are detected at runtime (e.g., Java checks upon storing into an array that the new entry value is a subtype of the array's original content type).

However, unintentional unsoundness in industrial type systems is rarer. The most notable example is Amin and Tate's discovery [2] that Java and Scala's handling of generic type constraints was rendered unsound by the fact that `null` is treated as inhabiting all reference types, making all type constraints (even nonsensical ones) satisfiable. Servetto et al. [8] first pointed out the issue publicly, though in the Q&A session after the talk, multiple Java designers were unsurprised — as Gilad Bracha put it, "Once I saw the title of your talk, I understood what the problem was."

Colin says: As I listened to the Q&A session again, James framed this as a question of whether two instances allocated in different contexts should be considered the same or not. I think that might be useful framing to bring up early here. Gilad also pointed out that (paraphrasing) the BETA people knew an inner class was a property of the instance, not its own class.

discuss C# casts with reified generics

Flix is a functional language for the JVM. The Flix compiler monomorphizes polymorphic functions, as Rust and MLton do. Flix has no subtyping. The Flix construct closest to a local class is the `new` expression, which creates an object of an unnamed class that extends a Java class or implements a Java interface . A Flix program can not name such a class, so a Flix program can not write a type test against it, as with the generated classes of C#. Thus the type test of Example 1 can not be written in Flix.

cite Flix FAQ, for monomorphization and no subtyping

cite Flix anonymous classes

The key issues at work in the interaction of method-local inner classes and generics arise from the interaction of (1) nominal dynamic downcasts, (2) design choices about what runtime information about generic parameters is both persisted *and checked* by casts, and (3) (not) tracking all dependencies on type parameters in the type system. Essentially, Java and Scala both treat `MyFoo` as a sort of existential type — that there exists some `X` such that `MyFoo` extends `Foo<X>`. However, treating `MyFoo` as a closed rather than generic type is equivalent to assuming that there is only one existential for all instances — associated with the declaration site — rather than potentially different existentials for different instances. This combined with the fact that the JVM is unable to dynamically check for different instantiations leads to the unsoundness. Our dynamic solution resolves this with a specific implementation of casts on reified generics.

Modern Java: under modern Java there is no separate dynamic solution; see the note on the dynamic approach in the introduction.

Our static solution effectively prohibits the de facto existential capture of the enclosing type parameter.

The decision to treat types like the `MyFoo` variants in our examples as closed existential types that depend on other choices of types is not without parallels in other languages. Haskell and Rust, for example, include associated types [3], where a typeclass has a type as a member. A yet closer analogue is Scala's path-dependent types , where types may be *members* of an individual object, rather than associated with an individual class. The code below adapts the troublesome example method using Scala's type members:

cite dot

```
trait Foo { type X; def x() : X; }
def m[Y](other: Any, newx: Y) : Foo2 =
    class MyFoo extends Foo { type X = Y; override def x() : X = newx; }
    newx match {
        case f: MyFoo => f
        case _ => MyFoo()
    }
```

This code compiles without warnings — and is not an issue. In this case the dependency of `MyFoo` instances on the type parameter used to call `m` is made explicit with a type member. But Scala now treats thes

8 Conclusions

write the conclusions

9 Draft Notes

Marco had this idea that casts may be avoided by using dynamic dispatch returning this. Can not work, since Meth local class is private, there is no way an external interface can mention it to provide the cast method. Of course, the method local is private to the body (this means also the method signature can not use it) This means no self:Foo method, nor self:T method of a generic class eventually instantiating T=Foo.

This means the core language DO NEED if(e instanceof T x) else

//nested funnelling: new A<X>() B<X> f()-> new B<X>() new A<X>() <K>B<X,K> f()-> new B<X,K>()

using 'extends' for declarations in top level classes and generic methods using 'funnel' for named inner classes new A<X funnel Object>() B<X> f()-> new B<X funnel Object>() new A<X funnel Object>() <K extends Object> B<X,K> f()-> new B<X funnel Object,K funnel Object>() or, if the bounds are not repeated new A<X funnel>() <K extends Object> B<X,K> f()-> new B<X funnel,K funnel>() Very funky syntax: we get new B<X funnel Object,K funnel Object>() but, during reduction, this can become we get new B<Person,K funnel Object>() where we have a mixture of concrete types and generic funnelling

Global class definition uses class C<X extends Bar>.. named method local class uses new C<X funnel>().. normal new: new C<Ts>()

```
e ::= x | e.m(es) | e.f | new C<Ts>(es) | new C<Fs>(){Ms} | if (e0 instanceof T x){ e1 }else{
T ::= C<Ts> | X
F ::= C<CCs> | X funnel// Important, can not be T | X funnel, otherwise we could have an X no
CC::= C<CCs> //types with zero free type variables
B::=  X extends T // Need T, not F, to be able to do things like (Java) class Bar<X,Y extends
```

```
Dec::= class C<Bs>{ Ms Ss} //assume standard constructor
S ::= T x; //field/slot
M ::= Sig; | Sig{ return e; }
Sig::= <Bs> T m(x1:T1,..xn:Tn)
G ::= x1:T1..xn:Tn
```

//we do not need Object to be able to write 'extends Object' //this is exactly as the distinction of checked and unchecked exceptions: formally it evaporates in syntacitc sugar where all is checked //and all methods 'throws Error, RTE' // Can assume the user provides their own Object type Note: an use of X is just X, a declaration of X is X extends T; a funnelling of X is X funnel Program is just many Dec plus an e typable in empty MARCO TODO: typechecking Decs;Bs;G|-e:T Decs|-Dec:ok Decs;T|-M:ok e –> e' Note, there are multiple semantics/ts: e instanceof CC //always clear e instanceof T with T not of form CC would not typecheck

e –> e' for Java with warn->err: just have CC in instanceof Java with magic reifined cast: just have T in instanceof (since reduction will replace(thus remember) generics Java with warn->off: AT RUN TIME instanceof only checks base class, ignores generic parameters AT COMPLILE TIME, instanceof can be err if the full type (including generics) is not potentially correlated)?? unobvious consider interfaces In prose, should explain that we won't flag cast of some type Foo to some type Bar, even though Java sometimes flags unrelated casts

Modern Java: these three semantics are replaced by the single one of Java 16+: erased run-time check, plus the checked-cast premise of rule IF-INSTANCEOF (Section 5.7). Unrelated types are rejected by it, as in Java.

<X extends List<Y funnel»
class Foo<X> class Getter<X funnel> <Y extends List<X» X getFirst(Y y) y.getOrElse(0,null)

<Y> .. return foo<Y,MyX>(); <Y,X extends List<Y» foo() return Foo<X funnel>()...
X ;

References

1 Ole Agesen, Stephen N Freund, and John C Mitchell. Adding type parameterization to the java language. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 49–65, 1997.

2 Nada Amin and Ross Tate. Java and scala's type systems are unsound: The existential crisis of null pointers. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, page 838–848, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2983990.2984004.

3 Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005.

4 James Gosling, Bill Joy, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. Java Language Specification, Java SE 23 Edition, 8 2024. URL: https://docs.oracle.com/javase/specs/jls/se23/html/index.html.

5 Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and gj. *ACM Trans. Program. Lang. Syst.*, 23(3):396–450, 5 2001. doi:10.1145/503502.503505.

6 Andrew Kennedy and Don Syme. Design and implementation of generics for the. net common language runtime. In *Proceedings of the ACM SIGPLAN 2001 conference on Programming language design and implementation*, pages 1–12, 2001.

7 Martin Odersky, Enno Runne, and Philip Wadler. Two ways to bake your pizza—translating parameterised types into java. In *Generic Programming: International Seminar on Generic Programming Dagstuhl Castle, Germany, April 27–May 1, 1998 Selected Papers*, pages 114–132. Springer, 2001.

8 Marco Servetto, Nick Webster, and Colin S. Gordon. Java method-local inner classes are unsound. Talk at UNSOUND 2024, co-located with SPLASH 2024, October 2024. Recording at `https://www.youtube.com/watch?v=DfkOpk733dQ`. URL: `https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound`.

9 The Rust Project. Closures: Anonymous functions that capture their environment, 7 2022. URL: `https://doc.rust-lang.org/book/ch13-01-closures.html`.